

But what do we mean by “changing the meaning of our program”? Just what *is* the meaning of a program? This is a somewhat philosophical question, and there are various ways of answering it that we won’t explore in detail here.² But the general point is that when we talk about referential transparency, it’s always with regard to some *context*. The context determines what sort of programs we’re interested in, and also how we assign meaning to our programs. Establishing this context is a choice; we need to decide what aspects of a program participate in that program’s meaning.

Let’s explore this subtlety a bit further.

14.3.1 What counts as a side effect?

Earlier, we talked about how the `eq` method is able to *observe* the side effect of object creation. Let’s look more closely at this idea of observable behavior and program meaning. It requires that we delimit what we consider observable and what we don’t. Take, for example, this method that has a definite side effect:

```
def timesTwo(x: Int) = {
  if (x < 0) println("Got a negative number")
  x * 2
}
```

If we replace `timesTwo(1)` with `2` in our program, we don’t have the same program in every respect. It may compute the same result, but we can say that the meaning of the program has changed. But this isn’t true for all programs that call `timesTwo`, nor for all notions of program equivalence.

We need to decide up front whether changes in standard output are something we care to observe—whether it’s part of the changes in behavior that *matter* in our context. In this case, it’s exceedingly unlikely that any other part of the program will be able to observe that `println` side effect occurring inside `timesTwo`.

Of course, `timesTwo` has a *hidden dependency* on the I/O subsystem. It requires access to the standard output stream. But as we’ve seen, most programs that we’d consider purely functional also require access to some of the underlying machinery of Scala’s environment, like being able to construct objects in memory and discard them. At the end of the day, we have to decide for ourselves which effects are important enough to track. We could use the `IO` monad to track `println` calls, but maybe we don’t want to bother. If we’re just using the console to do some temporary debug logging, it seems like a waste of time to track that. But if the program’s correct behavior depends in some way on what it prints to the console (like if it’s a UNIX command line utility), then we definitely want to track it.

This brings us to an essential point: tracking effects is a *choice* we make as programmers. It’s a value judgement, and there are trade-offs associated with how we choose. We can take it as far as we want. But as with the context of referential transparency, in Scala there’s a kind of standard choice. For example, it would be completely valid and

² See the chapter notes for links to further reading.